

Week9-It's raining functions!

Macarena López-Mayorga

2026-07-03

Functions

Structure

At the most basic level, what makes up a function in R? Let's take a look at a simple example:

```
MyFirstFunction <- function(){message("Always spin-down your antibodies!")}
```

The name of the function is defined by placing it on the left-side of the assignment arrow. To the right we place the function(), which conveys to R that it will be creating a function. Although currently empty, individual function arguments that will make up the inputs are placed within the parenthesis (). And finally, the actual code being run by the function is placed within the curly brackets {}.

When we run this code-block, we end up creating a function. We can see this appear within our local environment, appearing under the variables tab in the right-secondary sidebar.

Once we have created a function that is present in our environment, we are able to run/use it. In the case of the first function example, no inputs or arguments are needed (as there is nothing between the ()). For output, all that is present within the {} is code to send a message to console, which we will get back if we run the function by calling its name in our console.

```
MyFirstFunction()
```

Always spin-down your antibodies!

Arguments

In the case of our first function above, no input arguments were provided, meaning the function was entirely self-contained, allowing us to retrieve the code output by simply running the function name.

Lets modify this initial self-contained argument to allow us to provide an input argument to change the output message. To modify our function to accept input arguments, we decide on our argument name, and place it within the parenthesis (). From there, we can modify our line of code within the {} to achieve our desired outcome.

```
MySecondFunction <- function(BestFluorophore){message("Unquestionably, the best fluorophore is ", BestFluorophore)}
```

And we can check to make sure it is running correctly similar to how we would for any function we might encounter, by running the line of code

```
MySecondFunction(BestFluorophore = "APC")
```

Unquestionably, the best fluorophore is APC

Lets continue our roll and see if we can provide multiple arguments to our function. We can achieve this by separating individual function arguments within the () with commas. Don't forget to also make sure the argument you specified within the () is also accounted for within the {}.

```
MyThirdFunction <- function(BestFluorophore, WorstFluorophore){  
  message("Unquestionably, the best fluorophore is ", BestFluorophore)  
  message("And by far, the worst fluorophore is ", WorstFluorophore)  
}
```

```
MyThirdFunction(BestFluorophore = "APC", WorstFluorophore = "PerCP-Cy5.5")
```

Unquestionably, the best fluorophore is APC And by far, the worst fluorophore is PerCP-Cy5.5

Once we have a function set up that accepts multiple arguments, what happens if we forget to provide an argument? Let's intentionally forget "WorstFluorophore" and see.

```
MyThirdFunction(BestFluorophore="APC")
```

Unquestionably, the best fluorophore is APC Error in MyThirdFunction(): ! argument “WorstFluorophore” is missing, with no default

Show Traceback

As you see, we get back an error. The reason for this is that a function lives in relative isolation from the rest of our R session. It typically can only see what you provided as inputs, or what is spun up by the code within the function itself. We can also provide default values for our given arguments. For example, we could modify to provide WorstFluorophore default by setting a value equal to it within the ()

```
MyFourthFunction <- function(BestFluorophore, WorstFluorophore="PerCP-Cy5.5"){  
  message("Unquestionably, the best fluorophore is ", BestFluorophore)  
  message("And by far, the worst fluorophore is ", WorstFluorophore)  
}
```

Which would then allow for the partially incomplete function to fall back on the default argument.

```
MyFourthFunction(BestFluorophore="APC")
```

Unquestionably, the best fluorophore is APC And by far, the worst fluorophore is PerCP-Cy5.5

However, you are not locked in on your default option, as if you provide an actual argument, it will take preference over the provided default.

```
MyFourthFunction(BestFluorophore="APC", WorstFluorophore="Zombie Aqua")
```

Unquestionably, the best fluorophore is APC And by far, the worst fluorophore is Zombie Aqua

Iteration

Recalling back to Week 02, we ended up creating our first vector, which contained a list of fluorophores.

```
Fluorophores <- c("BV421", "FITC", "PE", "APC")  
str(Fluorophores)
```

```
chr [1:4] "BV421" "FITC" "PE" "APC"
```

Individual fluorophores from this vector can be provided as arguments to our function, by specifying their index location within the vector. For example:

```
::: {.cell}
```

```
MySecondFunction(BestFluorophore = Fluorophores[1])
```

```
:::
```

Unquestionably, the best fluorophore is BV421

If we wanted to check the output message for each item in our vector, one approach would be to copy-and-paste the line of code, and swap out the vector index position for each new line.

```
::: {.cell}
```

```
MySecondFunction(BestFluorophore = Fluorophores[2])
```

```
:::
```

Unquestionably, the best fluorophore is FITC

```
::: {.cell}
```

```
MySecondFunction(BestFluorophore = Fluorophores[4])
```

```
:::
```

Unquestionably, the best fluorophore is APC

While achievable, this ends up producing several lines of code. As you get further into the course, and more code is written to tidy individual dataframes, generate individual plots, having to copy-and-paste repeated blocks of text that only change based on the input values can become a chore. For this reason, there are a couple ways that we can sequentially pass through the objects within a vector or list to our function of interest, and retrieve the outputs that result.

For-loops A for-loop is one way to iterate through a vector. The syntax can look quite startling when encountered for a first time.

```
::: {.cell}
```

```
for (Name in Fluorophores){  
  MySecondFunction(BestFluorophore=Name)  
}
```

...

Unquestionably, the best fluorophore is BV421 Unquestionably, the best fluorophore is FITC
Unquestionably, the best fluorophore is PE Unquestionably, the best fluorophore is APC

In a way, it is similar to the “if” conditional we have encountered previously, with “for” being syntax highlighted. Immediately after for we find our parenthesis (), followed by {} that surround individual lines of code.

Within the (), the vector or list is being iterated through is placed after “in” (which also ends up being syntax highlighted). So in this case, we place our vector Fluorophores.

For what is placed before “in”, we basically provide an argument that will stand-in for the “i-nth” item in our vector. You will often see “i” used, but you can call it whatever you like, so we will use Name in this case. You will see this “i-nth” standin is then used in place of the value within our line of code present within the {}.

Consequently, when we run the for-loop, we get the following outputs: Unquestionably, the best fluorophore is BV421 Unquestionably, the best fluorophore is FITC Unquestionably, the best fluorophore is PE Unquestionably, the best fluorophore is APC

Can these outputs be saved? Yes they can, but some modification will be needed. Messages in R are kind of ephemeral and not saved, we could modify our original function from using messsages() to a more permanent paste0 statement to show-case this in a for-loop.

```
MySecondFunctionPartB <- function(BestFluorophore){  
  Message <- paste0("Unquestionably, the best fluorophore is ", BestFluorophore)  
  Message  
}
```

Secondly, we would need to save the output to something external that can be retrieved. This can be accomplished by setting up an external list using the list() function, and then adding the message to the list for each iteration within the for-loop brackets {}

```
MyOutputList <- list()  
  
SavedOutputs <- for (Name in Fluorophores){  
  InthMessage <- MySecondFunctionPartB(BestFluorophore=Name)  
  MyOutputList[[Name]] <- InthMessage  
}  
  
MyOutputList
```

\$BV421 [1] “Unquestionably, the best fluorophore is BV421”

```
$FITC [1] "Unquestionably, the best fluorophore is FITC"
```

```
$PE [1] "Unquestionably, the best fluorophore is PE"
```

```
$APC [1] "Unquestionably, the best fluorophore is APC"
```

So as you can see, for-loops are a bit complicated, but can be useful once you get the hang of the structure and syntax. One thing to note, is that they can be slow (especially if you are haven't predefined the list size), which is why opinions on their use will be contested by individual R developers.

purrr

When iterating through a vector or list to pass the values to a function, a commonly used alternative to for-loops are higher-order functions like the `walk()` and `map()` functions from the `purrr` package. These are the tidyverse equivalents to base R `apply()`, `lapply()`, etc. but with less nuanced argument syntax. We will use these frequently throughout the rest of the course.

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

```
filter, lag
```

The following objects are masked from 'package:base':

```
intersect, setdiff, setequal, union
```

Warning message: package 'dplyr' was built under R version 4.5.3

```
library(purrr)
```

Warning message: package 'purrr' was built under R version 4.5.3

```
walk(.x=Fluorophores, .f=MySecondFunction)
```

Unquestionably, the best fluorophore is BV421 Unquestionably, the best fluorophore is FITC
Unquestionably, the best fluorophore is PE Unquestionably, the best fluorophore is APC

As you can see, unlike for-loops, the additional structural elements provided are minimal. Within the (), our Fluorophore vector we want to iterate through is provided to the “x” argument, while to “f” we designate the function we want to provide the iterated outputs. Let’s see how walk() differs from its counterpart map()

```
Outputs <- map(.x=Fluorophores, .f=MySecondFunction)
```

Unquestionably, the best fluorophore is BV421 Unquestionably, the best fluorophore is FITC
Unquestionably, the best fluorophore is PE Unquestionably, the best fluorophore is APC

Outputs

```
[[1]] NULL
[[2]] NULL
[[3]] NULL
[[4]] NULL
```

In this case, we can see we get both the messages to console, but also individual empty (NULL) entries to a list-style object. This is partly due to the message() use we specified above for SecondFunction (which we corrected for with SecondFunctionPartB)

If we try to save walk() outputs using our modified PartB function, we quickly realize nothing is being saved, with only the input being recorded

```
Outputs <- walk(.x=Fluorophores, .f=MySecondFunctionPartB)
Outputs
```

```
[1] “BV421” “FITC” “PE” “APC”
```

The reason behind this is walk() is more behind the scenes, used under circumstances when you want something run, but don’t need the return. By contrast, returning things is the main focus of map().

Let’s see how walk() differs from its counterpart map()

```
Outputs <- map(.x=Fluorophores, .f=MySecondFunctionPartB)
Outputs
```

```
[[1]] [1] "Unquestionably, the best fluorophore is BV421"
```

```
[[2]] [1] "Unquestionably, the best fluorophore is FITC"
```

```
[[3]] [1] "Unquestionably, the best fluorophore is PE"
```

```
[[4]] [1] "Unquestionably, the best fluorophore is APC"
```

We will explore how these can be used more extensively in a bit.

Building Functions

Background

David Rach designed an automatic process of functions in R and we are going to work with these basis. So based on our initial idea, we know that we will need to get our .fcs files into a GatingSet, which we will then be able to iterate through.

As we iterate through these files, we will need to retrieve the description/keyword for volume that was encoded by the instrument, as well as any other useful metadata.

And we will need to apply gates (either through openCyto or flowGate), that would allow us to retrieve the counts for various cell populations we might be wanting to retrieve concentration info for.

Let's go ahead and tackle these in turn, re-using code that we have encountered over the last several weeks

Getting started, as always, we will need to load any libraries we will be using extensively.

```
library(flowWorkspace)
```

As part of improvements to flowWorkspace, some behavior of GatingSet objects has changed. For details, please read the section titled "The cytoframe and cytoset classes" in the package vignette:

```
vignette("flowWorkspace-Introduction", "flowWorkspace")
```

```
library(flowGate)
```

Loading required package: ggcyto Loading required package: ggplot2 Loading required package: flowCore Loading required package: ncdfflow Loading required package: BH Warning message: package 'ggplot2' was built under R version 4.5.3

```
library(ggplot2)
library(ggcyto)
```

And then we need to identify our fcs files

```
StorageLocation <- file.path("C:/Users/Macarena/MacarenaLM/notes/Week9-It's raining functions!")
fcs_files <- list.files(StorageLocation, pattern=".fcs", full.names=TRUE)
```

```
fcs_files
```

```
[1] "C:/Users/Macarena/MacarenaLM/notes/Week9-It's raining functions!/data/Cell-
Counts4L_AB_04-INF124-7-00_01.fcs" [2] "C:/Users/Macarena/MacarenaLM/notes/Week9-
It's raining functions!/data/CellCounts4L_AB_04-INF149-4-00_01.fcs" [3] "C:/Users/Macarena/MacarenaLM/notes/Week9-It's raining functions!/data/CellCounts4L_AB_04-INF169-6-00_01.fcs"
[4] "C:/Users/Macarena/MacarenaLM/notes/Week9-It's raining functions!/data/Cell-
Counts4L_AB_05-INF019-0-00_01.fcs" [5] "C:/Users/Macarena/MacarenaLM/notes/Week9-
It's raining functions!/data/CellCounts4L_AB_05-INF032-5-00_01.fcs" [6] "C:/Users/Macarena/MacarenaLM/notes/Week9-It's raining functions!/data/CellCounts4L_AB_05-INF180-9-00_01.fcs"
[7] "C:/Users/Macarena/MacarenaLM/notes/Week9-It's raining functions!/data/Cell-
Counts4L_AB_06-INF155-1-00_01.fcs" [8] "C:/Users/Macarena/MacarenaLM/notes/Week9-
It's raining functions!/data/CellCounts4L_AB_06-INF158-4-00_01.fcs" [9] "C:/Users/Macarena/MacarenaLM/notes/Week9-It's raining functions!/data/CellCounts4L_AB_06-INF159-5-00_01.fcs"
```

Before proceeding to add them into a GatingSet

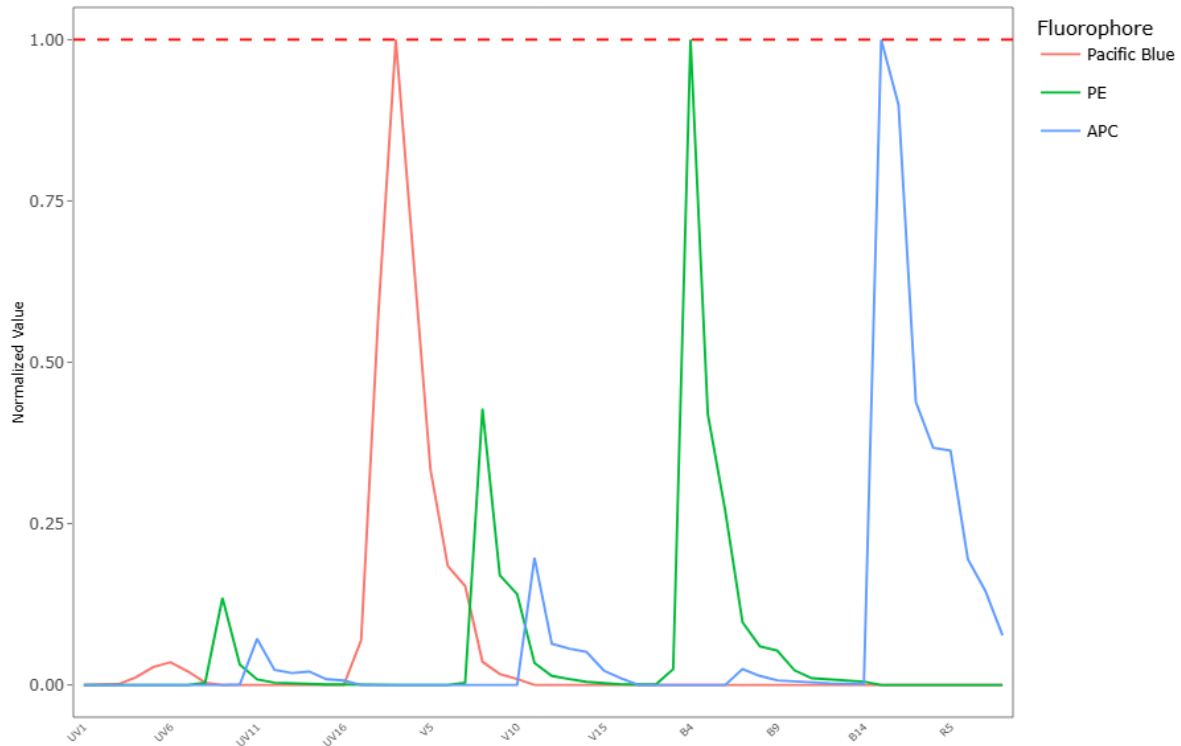
```
MyCytoset <- load_cytoset_from_fcs(fcs_files, transformation=FALSE, truncate_max_range=FALSE)
MyGatingSet <- GatingSet(MyCytoset)
```

For this particular counting fluorophore panel, only 3 antibodies were used. Since the goal was to quickly acquire the .fcs files before passing off to R, the files were never unmixed, and are still in a raw. fcs format. So we will need to figure out the peak detectors for APC, PE, and Pacific Blue on a 4-laser Cytex Aurora.

```
remotes::install_github("DavidRach/Luciernaga")
```

```
install.packages("plotly")
```

```
Plots <- Luciernaga::QC_ReferenceLibrary(FluorNameContains="APC$|PE$|Pacific Blue",
  NumberDetectors=54, returnPlots=TRUE)[[2]]
plotly::ggplotly(Plots)
```



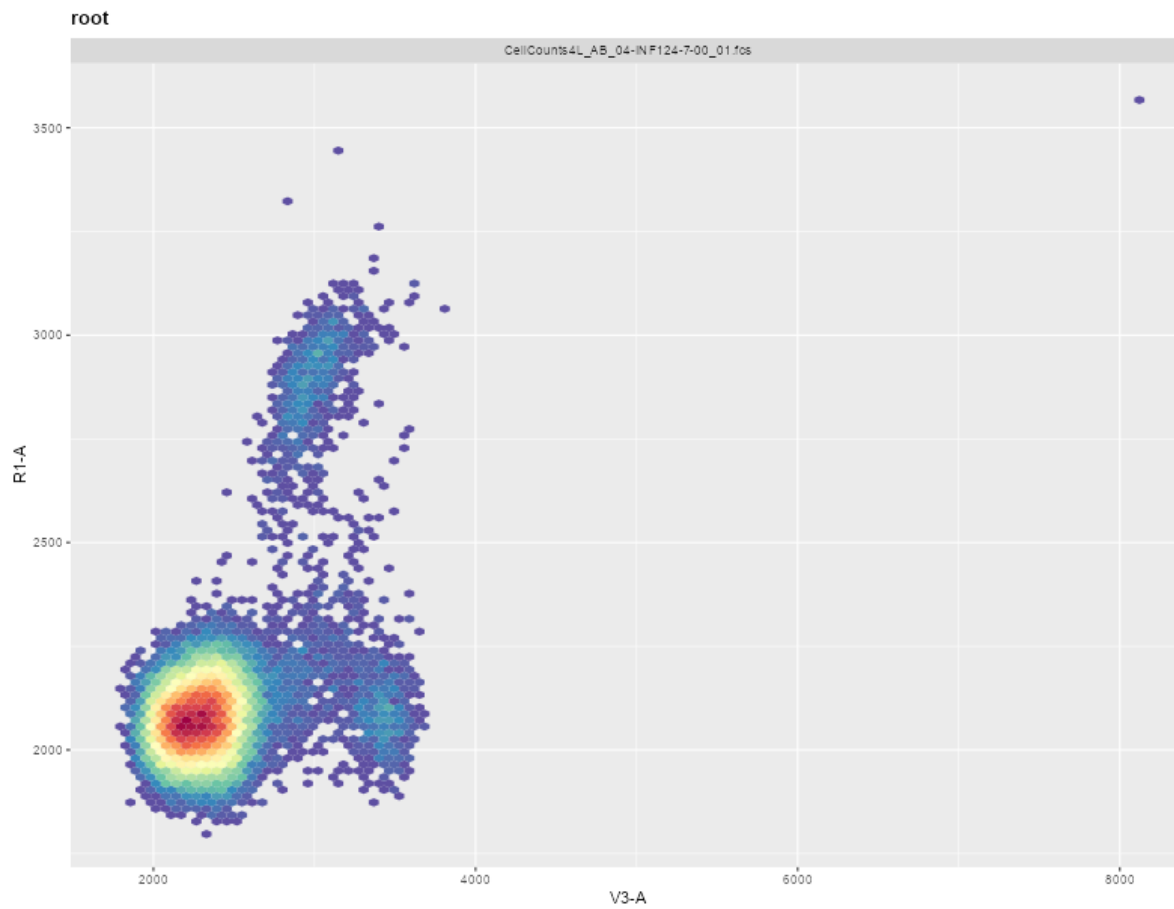
So the equivalent raw detectors we should be paying attention to are going to be V3 (CD19+), B4 (CD45+), and R1 (CD14+) (not possible to see in the picture) Since scale can vary dramatically for individual fluorophores, we might as well and go ahead and transform those three detectors while we are here.

```
MainDetectorsOnly <- c("V3-A", "B4-A", "R1-A")

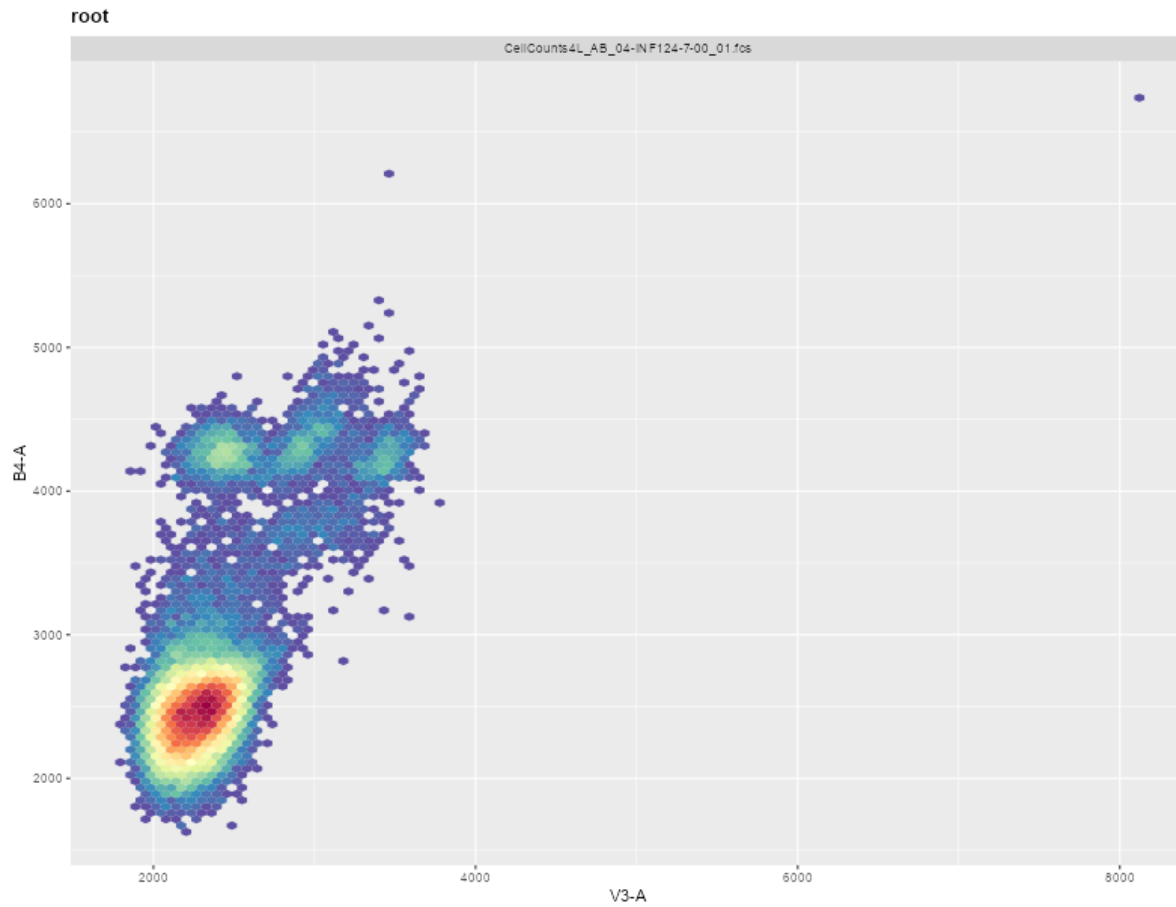
Biexponential <- flowjo_biexp_trans(channelRange=4096, maxValue=262144,
  pos=4.5, neg=2, widthBasis=-500)
MyBiexTransform <- transformerList(MainDetectorsOnly, Biexponential)
transform(MyGatingSet, MyBiexTransform)
```

And a quick-plot check to make sure the transformation looks reasonable enough

```
ggcyto(MyGatingSet[[1]], subset="root",
  aes(x="V3-A", y="R1-A")) + geom_hex(bins=100)
```



```
ggcyto(MyGatingSet[[1]], subset="root",  
  aes(x="V3-A", y="B4-A")) + geom_hex(bins=100)
```



```
ggcyto(MyGatingSet[[1]], subset="root",  
  aes(x="R1-A", y="B4-A")) + geom_hex(bins=100)
```



With that done, let's add some gates to separate out CD45+ cells, and below those cells identify CD14+ (mainly monocytes), CD19+ (B cells), and CD14-CD19- (mostly T cells and NK cells for cord blood). My automated version uses openCyto with gate constraints to handle the variability, but for this example, let's reuse the flowGate approach using tribble function for quicker setup.

```
GatingTable <- tibble::tribble(
  ~filterId,    ~dims,                                ~subset,
  "CD45",       list("B4-A", "SSC-A"),                "root",
  "CD19",       list("V3-A", "SSC-A"),                "CD45",
  "CD14",       list("R1-A", "SSC-A"),                "CD45",
  "TandNKcells", list("V3-A", "SSC-A"),                "CD45",
)

flowGate::gs_apply_gating_strategy(MyGatingSet, gating_strategy = GatingTable)
```

Listening on <http://127.0.0.1:6900> done!

Listening on http://127.0.0.1:6900 done!

Listening on http://127.0.0.1:6900 Warning: Error in if: argument is of length zero 151: getDims 150: catch 149: value[[3L]] 148: tryCatchOne 147: tryCatchList 146: tryCatch 145: do 144: hybrid_chain 143: 121: drawReactive 108: renderFunc 106: output\$plot1 8: shiny::runApp 7: .f 3: pmap__ 2: purrr::pmap 1: flowGate::gs_apply_gating_strategy done!

Listening on http://127.0.0.1:6900 done! [[1]] [[1]]\$Gate Polygonal gate 'CD45' with 22 vertices in dimensions B4-A and SSC-A

```
[[1]]$Bins [1] 256
```

```
[[1]]$Scaling [1] "unused"
```

[[2]] [[2]]\$Gate Polygonal gate 'CD19' with 11 vertices in dimensions V3-A and SSC-A

```
[[2]]$Bins [1] 256
```

```
[[2]]$Scaling [1] "unused"
```

[[3]] [[3]]\$Gate Polygonal gate 'CD14' with 12 vertices in dimensions R1-A and SSC-A

```
[[3]]$Bins [1] 256
```

```
[[3]]$Scaling [1] "unused"
```

[[4]] [[4]]\$Gate Polygonal gate 'TandNKcells' with 15 vertices in dimensions V3-A and SSC-A

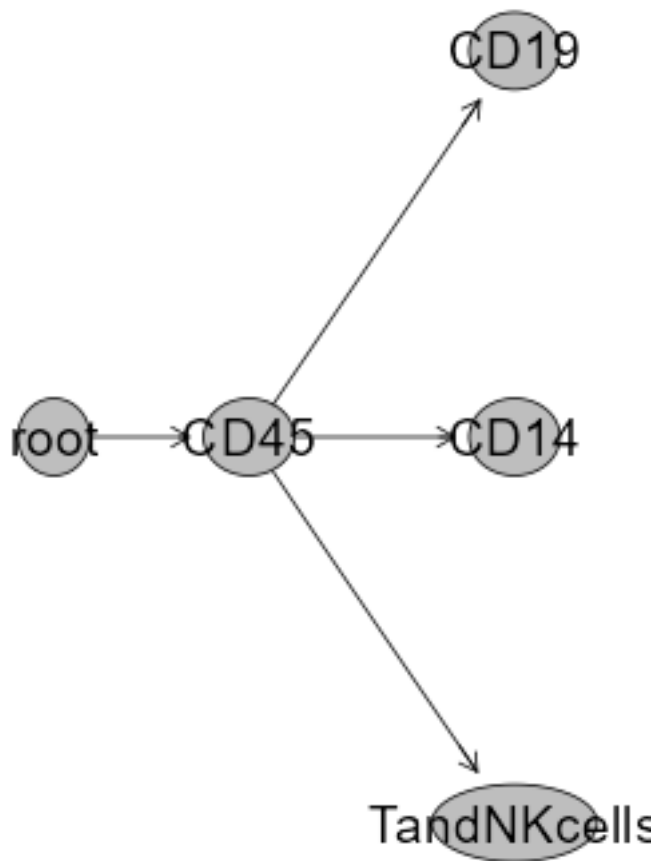
```
[[4]]$Bins [1] 256
```

```
[[4]]$Scaling [1] "unused"
```

We have add a general CD45+ gate (i.e. those that I need actual concentration for), a general CD19+ gate (B cells), a general CD14+ (Monocytes), and a general gate for likely T and NK cells (one after the other, in this exact order).

Ending up with a GatingSet that looks like this.

```
plot(MyGatingSet)
```



I'm going to save the GatingSet to load it in the future when I decide to continue with the notes:

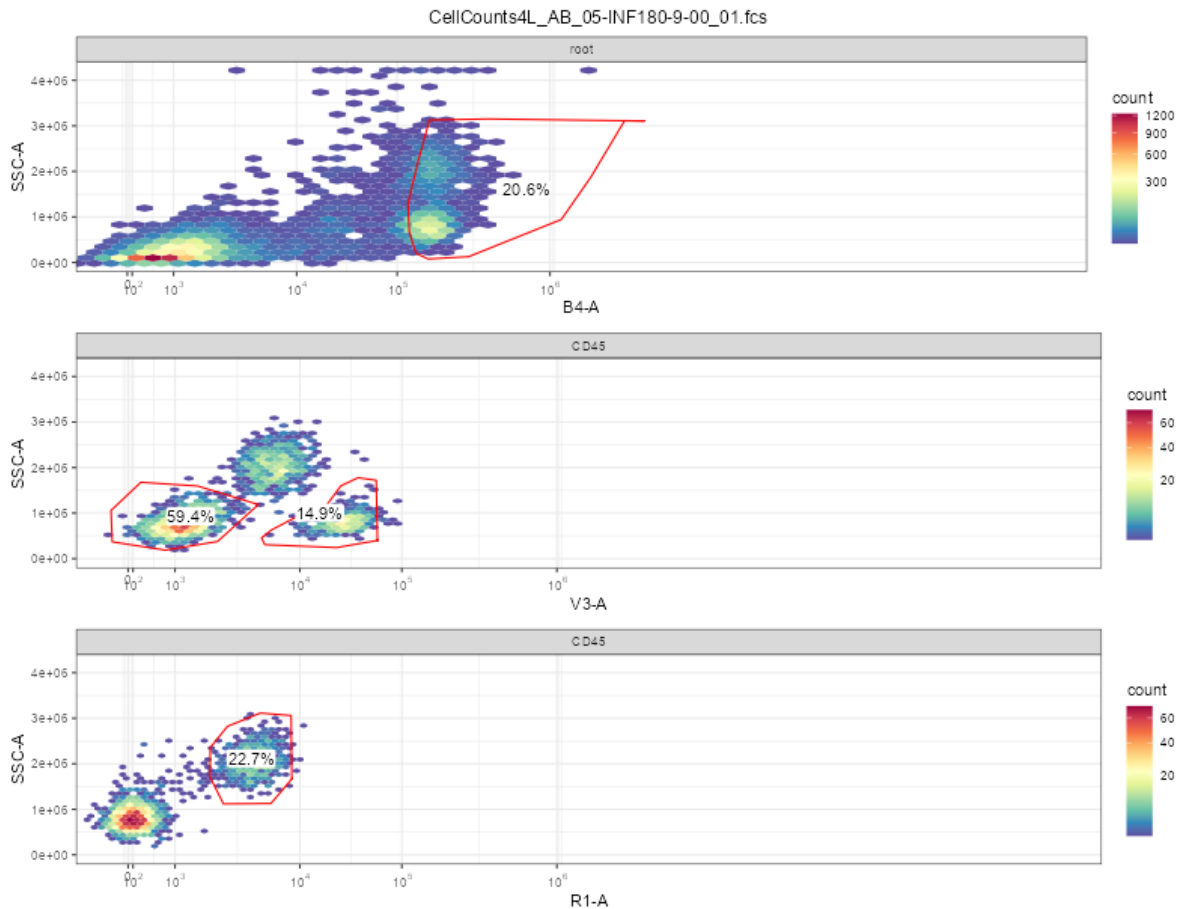
```
StorageGatingSet <- file.path("C:/Users/Macarena/MacarenaLM/notes/Week9-It's raining function  
save_gs(MyGatingSet, StorageGatingSet)
```

Done To reload it, use 'load_gs' function

```
StorageGatingSet <- file.path("C:/Users/Macarena/MacarenaLM/notes/Week9-It's raining function  
MyGatingSet <- load_gs(StorageGatingSet)
```

And with that done, let's quickly check the gates using ggcyto autoplot() function, switching between specimen index values to double check.

```
ggcyto::autoplot(MyGatingSet[[6]]) + theme_bw()
```



Building

Alright, lets start by creating the first layer of our function

```
CellConcentration <- function(){}
```

At the moment, our function is literally nothing but a wrapper, but when we run the line of code, it is now present under the variables tab. Lets start by considering what we want to do. We have each of our .fcs files currently contained within a GatingSet object. Ultimately, we want to be able to retrieve the cell concentration from each of these files.

To do so, lets go ahead and plan that we will use the `map()` function to iterate our way through each specimen of the GatingSet object

```
pData(MyGatingSet)$name
```

```
[1] "CellCounts4L_AB_04-INF124-7-00_01.fcs" "CellCounts4L_AB_04-INF149-4-00_01.fcs" "CellCounts4L_AB_04-INF169-6-00_01.fcs" [4] "CellCounts4L_AB_05-INF019-0-00_01.fcs" "CellCounts4L_AB_05-INF032-5-00_01.fcs" "CellCounts4L_AB_05-INF180-9-00_01.fcs" [7] "CellCounts4L_AB_06-INF155-1-00_01.fcs" "CellCounts4L_AB_06-INF158-4-00_01.fcs" "CellCounts4L_AB_06-INF159-5-00_01.fcs"
```

Since when we iterate with `map()` we need to specify a `.x` argument, let's go ahead and set that as our argument within the `()`.

```
CellConcentration <- function(x){}
```

And to make our lives a little easier, we should provide some function documentation as we go, so that future us several months from now can remember what under-caffenaited us today did. In R, this typically is done via what is known as a roxygen2 skeleton, which looks like this.

```
#' This function will figure out the cell concentration
#' for a given gated population in an .fcs file
#'
#' @param x A GatingSet object (typically iterated in)
#'
CellConcentration <- function(x){}
```

In a Roxygen skeleton, we use a modified comment (`#'`) format to identify this as documentation. The start is a generic description for our function. The `@` param is used to list the individual argument, and a short description of what it does.

Alright, with that bit of documentation done so that we can contextualize what we did later, let's go back and think mechanics. We will be passing our function through `map()`, so our line of code that we will run will look something like this

```
purrr::map(.x=MyGatingSet, .f=CellConcentration)
```

```
[[1]] NULL
```

```
[[2]] NULL
```

```
[[3]] NULL
```

```
[[4]] NULL
```

```
[[5]] NULL
```

```
[[6]] NULL
```

```
[[7]] NULL
```

```
[[8]] NULL
```

```
[[9]] NULL
```

As map is triggered, we will basically be indexing our way through MyGatingSet, similar to if we ran the lines of code like this

```
MyGatingSet[[1]]
```

Sample: CellCounts4L_AB_04-INF124-7-00_01.fcs GatingHierarchy with 5 gates

```
MyGatingSet[[2]]
```

Sample: CellCounts4L_AB_04-INF149-4-00_01.fcs GatingHierarchy with 5 gates

Let's go ahead and add something inside our function so we can get a readout of what is happening inside the function. Let's use our old-friend class() to confirm what is being passed in via map().

```
#' This function will figure out the cell concentration
#' for a given gated population in an .fcs file
#'
#' @param x A GatingSet object (typically iterated in)
#'
CellConcentration <- function(x){class(x)}
```

```
purrr::map(.x=MyGatingSet, .f=CellConcentration)
```

```
[[1]] [1] "GatingHierarchy" attr("package") [1] "flowWorkspace"
```

```
[[2]] [1] "GatingHierarchy" attr("package") [1] "flowWorkspace"
```

```
[[3]] [1] "GatingHierarchy" attr("package") [1] "flowWorkspace"
```

```
[[4]] [1] "GatingHierarchy" attr("package") [1] "flowWorkspace"
```

```
[[5]] [1] "GatingHierarchy" attr("package") [1] "flowWorkspace"
```

```
[[6]] [1] "GatingHierarchy" attr("package") [1] "flowWorkspace"
```

```
[[7]] [1] "GatingHierarchy" attr("package") [1] "flowWorkspace"
```

```
[[8]] [1] "GatingHierarchy" attr("package") [1] "flowWorkspace"
```

```
[[9]] [1] "GatingHierarchy" attr(,"package") [1] "flowWorkspace"
```

So passing through map, we are ending up with individual "GatingHierarchy" (GatingSet = 1).

So with a GatingHierarchy object inside, let's turn to seeing what flowWorkspace functions we can use to get our desired outputs out of the description/keyword list.

```
#' This function will figure out the cell concentration
#' for a given gated population in an .fcs file
#'
#' @param x A GatingSet object (typically iterated in)
#'
CellConcentration <- function(x){keyword(x)}
```

Let's also set map() to only target the first specimen in MyGatingSet for now

```
KeywordList <- purrr::map(.x=MyGatingSet[1], .f=CellConcentration)
head(KeywordList[[1]], 20)
```

```
$FCSversion [1] "3.0999999"
`BEGINDATA` [1] "15152"
`ENDDATA` [1] "8436079"
`BEGINANALYSIS` [1] "0"
`ENDANALYSIS` [1] "0"
`BEGINTEXT` [1] "0"
`ENDSTEXT` [1] "0"
`MODE` [1] "L"
`BYTEORD` [1] "1,2,3,4"
`DATATYPE` [1] "F"
`PAR` [1] "61"
`TOT` [1] "34512"
`NEXTDATA` [1] "0"
`TIMESTEP` [1] "0.0001"
`DATE` [1] "30-Jul-2025"
```

```
`BTIM` [1] "13:48:10.76"
```

```
`ETIM` [1] "13:48:37.88"
```

```
`OP` [1] "David Rach"
```

```
`INST` [1] "UMBC"
```

```
`CYT` [1] "Aurora"
```

Alright, so we have access to the internal .fcs file keywords, similar to what we saw during Week 03. We can check through the full list and see what might be useful for us for this targeted problem, and within the function isolate them to their own respective variables for subsequent use.

```
#' This function will figure out the cell concentration
#' for a given gated population in an .fcs file
#'
#' @param x A GatingSet object (typically iterated in)
#'
CellConcentration <- function(x){Volume <- keyword(x)$`$VOL`}
```

```
purrr::map(.x=MyGatingSet[1], .f=CellConcentration)
```

```
[[1]] [1] "30.99"
```

As you can see, since we have changed the internal code within the function, we are now getting back the value recorded for acquired volume when our function is run

```
#' This function will figure out the cell concentration
#' for a given gated population in an .fcs file
#'
#' @param x A GatingSet object (typically iterated in)
#'
CellConcentration <- function(x){
  Date <- keyword(x)$`$DATE` # Day-Month-Year
  InstrumentSerial <- keyword(x)$`$CYTSN`
  TotalFileEvents <- keyword(x)$`$TOT`
  Specimen <- keyword(x)$GUID
  StartTime <- keyword(x)$`$BTIM`
  EndTime <- keyword(x)$`$ETIM`
  Volume <- keyword(x)$`$VOL`
}
```

```
purrr::map(.x=MyGatingSet[1], .f=CellConcentration)
```

```
[[1]] [1] "30.99"
```

You will notice however that despite finding all these other keywords, we are only getting back the volume as an output. The reason for this is a function returns by default the last object created within it. We can change this by setting a `return()` value equal to something else.

```
#' This function will figure out the cell concentration
#' for a given gated population in an .fcs file
#'
#' @param x A GatingSet object (typically iterated in)
#'
CellConcentration <- function(x){
  Date <- keyword(x)$`$DATE` # Day-Month-Year
  InstrumentSerial <- keyword(x)$`$CYTSN`
  TotalFileEvents <- keyword(x)$`$TOT`
  Specimen <- keyword(x)$GUID
  StartTime <- keyword(x)$`$BTIM`
  EndTime <- keyword(x)$`$ETIM`
  Volume <- keyword(x)$`$VOL`

  return(InstrumentSerial)
}
```

```
purrr::map(.x=MyGatingSet[1], .f=CellConcentration)
```

```
[[1]] [1] "V0333"
```

In this case, we are just retrieving individual bits and pieces. Ideally, we will want to cobble these together into a “data.frame” row or column so that we get everything back in a single go, and have a dataset that can then be used for tidyverse filtering, plotting, etc.

One thing to notice, there is a Total Events (TOT) keyword.

```
#' This function will figure out the cell concentration
#' for a given gated population in an .fcs file
#'
#' @param x A GatingSet object (typically iterated in)
#'
CellConcentration <- function(x){
  Date <- keyword(x)$`$DATE` # Day-Month-Year
  InstrumentSerial <- keyword(x)$`$CYTSN`
```

```

    TotalFileEvents <- keyword(x)$`$TOT`
    Specimen <- keyword(x)$GUID
    StartTime <- keyword(x)$`$BTIM`
    EndTime <- keyword(x)$`$ETIM`
    Volume <- keyword(x)$`$VOL`

    return(TotalFileEvents)
}

```

```
purrr::map(.x=MyGatingSet[1], .f=CellConcentration)
```

```
[[1]] [1] "34512"
```

This however corresponds to every event in the .fcs file, including doublets, debris, and miscellaneous cell populations, nothing specific to our individually drawn gates.

What we can do instead, is use a flowWorkspace function we learned about during Week 05, namely the `gs_pop_get_data()` function.

```

MySubset <- gs_pop_get_data(MyGatingSet[[1]], "root")
nrow(MySubset)[1]

```

```
[1] 34512
```

The main purpose for `gs_pop_get_data()` is to reach through the pointer and retrieve the data associated with a particular gate. We can then assess this subsetted data using our regular R functions.

You may have noticed the subset argument. We can switch it out for the respective gate names to retrieve the counts for cells present within each.

```

TheSubset <- gs_pop_get_data(MyGatingSet[1], "CD45")
nrow(TheSubset)

```

```
$CellCounts4L_AB_04-INF124-7-00_01.fcs [1] 2779
```

```

TheSubset <- gs_pop_get_data(MyGatingSet[1], "CD14")
nrow(TheSubset)

```

```
$CellCounts4L_AB_04-INF124-7-00_01.fcs [1] 706
```

```
TheSubset <- gs_pop_get_data(MyGatingSet[1], "CD19")
nrow(TheSubset)
```

```
$CellCounts4L_AB_04-INF124-7-00_01.fcs [1] 545
```

```
TheSubset <- gs_pop_get_data(MyGatingSet[1], "TandNKcells")
nrow(TheSubset)
```

```
$CellCounts4L_AB_04-INF124-7-00_01.fcs [1] 1369
```

As you can see, we can access the counts for the cells found within the respective gate by providing the gate name as the second argument. Let's modify our function, to add subset argument

```
#' This function will figure out the cell concentration
#' for a given gated population in an .fcs file
#'
#' @param x A GatingSet object (typically iterated in)
#' @param subset The gate from which to retrieve cell counts from
#'
CellConcentration <- function(x, subset){
  Date <- keyword(x)$`$DATE` # Day-Month-Year
  InstrumentSerial <- keyword(x)$`$CYTSN`
  TotalFileEvents <- keyword(x)$`$TOT`
  Specimen <- keyword(x)$GUID
  StartTime <- keyword(x)$`$BTIM`
  EndTime <- keyword(x)$`$ETIM`
  Volume <- keyword(x)$`$VOL`

  EventsInTheGate <- gs_pop_get_data(x, subset)
  Cells <- nrow(EventsInTheGate)

  return(Cells)
}
```

```
purrr::map(.x=MyGatingSet[1], .f=CellConcentration, subset="CD45")
```

```
[[1]] [[1]]$CellCounts4L_AB_04-INF124-7-00_01.fcs [1] 2779
```

Alright, we have now found all the bits and pieces we will likely need to estimate cell concentration. Let's start by making sure that Volume is switched over to numeric (as keywords can often be stored as character values)

```

#' This function will figure out the cell concentration
#' for a given gated population in an .fcs file
#'
#' @param x A GatingSet object (typically iterated in)
#' @param subset The gate from which to retrieve cell counts from
#'
CellConcentration <- function(x, subset){
  Date <- keyword(x)$`$DATE` # Day-Month-Year
  InstrumentSerial <- keyword(x)$`$CYTSN`
  TotalFileEvents <- keyword(x)$`$TOT`
  Specimen <- keyword(x)$GUID
  StartTime <- keyword(x)$`$BTIM`
  EndTime <- keyword(x)$`$ETIM`

  Volume <- keyword(x)$`$VOL`
  Volume <- as.numeric(Volume)

  EventsInTheGate <- gs_pop_get_data(x, subset)
  Cells <- nrow(EventsInTheGate)

  return(Cells)
}

```

Next, remembering our $C1V1 = C2V2$, we can set up the formula

```

#|eval: false
#' This function will figure out the cell concentration
#' for a given gated population in an .fcs file
#'
#' @param x A GatingSet object (typically iterated in)
#' @param subset The gate from which to retrieve cell counts from
#'
CellConcentration <- function(x, subset){
  Date <- keyword(x)$`$DATE` # Day-Month-Year
  InstrumentSerial <- keyword(x)$`$CYTSN`
  TotalFileEvents <- keyword(x)$`$TOT`
  Specimen <- keyword(x)$GUID
  StartTime <- keyword(x)$`$BTIM`
  EndTime <- keyword(x)$`$ETIM`

  Volume <- keyword(x)$`$VOL`
  Volume <- as.numeric(Volume)

```

```

EventsInTheGate <- gs_pop_get_data(x, subset)
Cells <- nrow(EventsInTheGate)[[1]] #[[1]] since the output is in a list

Concentration <- (Cells*1000)/Volume

return(Concentration)
}

```

And let's check across our specimens:

```
purrr::map(.x=MyGatingSet, .f=CellConcentration, subset="CD19")
```

```

[[1]] [1] 17586.32
[[2]] [1] 10281.28
[[3]] [1] 6588.77
[[4]] [1] 13324.58
[[5]] [1] 6625.326
[[6]] [1] 12072.89
[[7]] [1] 15934.07
[[8]] [1] 15517.82
[[9]] [1] 4591.012

```

In the case for our counting tubes, after staining they were diluted 1:100. So we need to add a dilution multiplier to the above. We can set that as our third argument.

```

#' This function will figure out the cell concentration
#' for a given gated population in an .fcs file
#'
#' @param x A GatingSet object (typically iterated in)
#' @param subset The gate from which to retrieve cell counts from
#' @param dilutionMultiplier Multiplies the estimated concentration
#' to bring correct digits.
#'
CellConcentration <- function(x, subset, dilutionMultiplier){
  Date <- keyword(x)$`$DATE` # Day-Month-Year
  InstrumentSerial <- keyword(x)$`$CYTSN`
  TotalFileEvents <- keyword(x)$`$TOT`

```

```

Specimen <- keyword(x)$GUID
StartTime <- keyword(x)$`$BTIM`
EndTime <- keyword(x)$`$ETIM`

Volume <- keyword(x)$`$VOL`
Volume <- as.numeric(Volume)

EventsInTheGate <- gs_pop_get_data(x, subset)
Cells <- nrow(EventsInTheGate)[[1]] #[[1]] since the output is in a list

Concentration <- (Cells*1000)/Volume
Concentration <- Concentration*dilutionMultiplier

return(Concentration)
}

```

```

purrr::map(.x=MyGatingSet, .f=CellConcentration, subset="CD19", dilutionMultiplier=100)

```

```

[[1]] [1] 1758632
[[2]] [1] 1028128
[[3]] [1] 658877
[[4]] [1] 1332458
[[5]] [1] 662532.6
[[6]] [1] 1207289
[[7]] [1] 1593407
[[8]] [1] 1551782
[[9]] [1] 459101.2

```

```

#' This function will figure out the cell concentration
#' for a given gated population in an .fcs file
#'
#' @param x A GatingSet object (typically iterated in)
#' @param subset The gate from which to retrieve cell counts from
#' @param dilutionMultiplier Multiplies the estimated concentration
#' to bring correct digits.
#'
CellConcentration <- function(x, subset, dilutionMultiplier){

```

```

Date <- keyword(x)$`$DATE` # Day-Month-Year
InstrumentSerial <- keyword(x)$`$CYTSN`
TotalFileEvents <- keyword(x)$`$TOT`
Specimen <- keyword(x)$GUID
StartTime <- keyword(x)$`$BTIM`
EndTime <- keyword(x)$`$ETIM`

Volume <- keyword(x)$`$VOL`
Volume <- as.numeric(Volume)

EventsInTheGate <- gs_pop_get_data(x, subset)
Cells <- nrow(EventsInTheGate)[[1]] #[[1]] since the output is in a list

Concentration <- (Cells*1000)/Volume
Concentration <- Concentration*dilutionMultiplier

Data <- data.frame(
  Date=Date,
  InstrumentSerial=InstrumentSerial,
  Specimen=Specimen,
  StartTime=StartTime,
  EndTime=EndTime,
  Volume=Volume,
  Cells=Cells,
  Concentration=Concentration
)

return(Data)
}

```

```
purrr::map(.x=MyGatingSet[1], .f=CellConcentration, subset="CD19", dilutionMultiplier=100)
```

	Date	InstrumentSerial	Specimen	StartTime	EndTime	Vol
1	30-Jul-2025	V0333	CellCounts4L_AB_04-INF124-7-00_01.fcs	13:48:10.76	13:48:37.88	30.99
545		1758632				

And we can use the dplyr package `bind_rows()` to assemble all our outputs for the individual .fcs files

```
Dataset <- purrr::map(.x=MyGatingSet, .f=CellConcentration, subset="CD19",
dilutionMultiplier=100) |> bind_rows()
```

Dataset

	Date	InstrumentSerial		Specimen	StartTime	EndTime	Vol
1	30-Jul-2025	V0333	CellCounts4L_AB_04-INF124-7-00_01.fcs	13:48:10.76	13:48:37.88		
30.99	545	1758631.8	2	30-Jul-2025	V0333	CellCounts4L_AB_04-INF149-4-00_01.fcs	
13:33:43.49	13:34:09.95	30.93	318	1028128.0	3	30-Jul-2025	V0333
				CellCounts4L_AB_04-INF169-6-00_01.fcs	13:40:42.92	13:41:10.01	30.81
203	658877.0	4	04-Aug-2025	V0333	CellCounts4L_AB_05-INF019-0-00_01.fcs	13:41:23.34	13:41:50.51
30.47	406	1332458.2	5	04-Aug-2025	V0333	CellCounts4L_AB_05-INF032-5-00_01.fcs	13:35:13.99
13:35:41.15	30.64	203	662532.6	6	04-Aug-2025	V0333	CellCounts4L_AB_05-INF180-9-00_01.fcs
13:48:19.41	13:48:46.83	30.73	371	1207289.3	7	06-Aug-2025	V0333
				CellCounts4L_AB_06-INF155-1-00_01.fcs	13:45:36.08	13:46:03.30	30.94
493	1593406.6	8	06-Aug-2025	V0333	CellCounts4L_AB_06-INF158-4-00_01.fcs	13:39:27.95	13:39:54.12
30.03	466	1551781.6	9	06-Aug-2025	V0333	CellCounts4L_AB_06-INF159-5-00_01.fcs	13:52:09.56
13:52:36.75	30.93	142	459101.2				

And now with our dataset complete, we can save it out for later use.

```
StorageLocation <- file.path("C:/Users/Macarena/MacarenaLM/notes/Week9-It's raining functions")
StorageDataset <- file.path(StorageLocation, "CellConcentrationsForTodaysExperiment.csv")
write.csv(Dataset, StorageDataset, row.names=FALSE)
```